



TRIBHUVAN UNIVERSITY

FACULTY OF HUMANITIES AND SOCIAL SCIENCE

A

Project Report On

Phishing Link Detection System

Submitted to

Department of Computer Application

Southwestern State College, Basundhara, Kathmandu

In partial fulfillment of the requirements for the Bachelor in Computer Application

Submitted By

Name: Rojina Ghimire

TU Reg. No: 6-2-530-26-2020

November, 2024

Under the Supervision of

Kiran Ghimire



Tribhuvan University
Faculty of Humanities and Social Science
Southwestern State College

SUPERVISOR'S RECOMMENDATION

We hereby recommend that this project is prepared by **Rojina Ghimire(6-2-530-26-2020)** under supervision by **Mr. Kiran Ghimire** entitled “**Phishing Link Detection System**” in partial fulfillment of the requirements for the degree of Bachelor of Computer Application be processed for the evaluation.

Mr. Kiran Ghimire Supervisor

Head of Department of BCA Southwestern State College



Tribhuvan University
Faculty of Humanities and Social Science
Southwestern State College

LETTER OF APPROVAL

This is to certify that this project is prepared by **Rojina Ghimire (6-2-530-26-2020)** entitled “**Phishing Link Detection System**” in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

Kiran Ghimire
Project Supervisor
BCA Department SWSC

Bijay Babu Regmi
Internal Supervisor

Bikash Parajuli
Internal Supervisor

External Examiner
Tribhuvan University

ACKNOWLEDGMENT

This is an attempt to present the project entitled "**Phishing Link Detection System**" prepared for partial fulfillment of the requirement for the Degree of Bachelor in computer application (BCA) is an outcome of continuous and immeasurable cooperation and support of several hands. I express my sincere honor and special sense of gratitude to my academic supervision **Mr. Kiran Ghimire**. I would also like to thank all lecturer and friends for their generous guidance, thoughtful encouragement and brilliant insight throughout this work.

With Respect,
Rojina Ghimire
(6-2-530-26-2020)

ABSTRACT

Phishing websites are a pervasive threat, often used by cybercriminals to deceive users into disclosing sensitive information by mimicking legitimate sites. This project presents a phishing link detection system designed to protect users from falling victim to these attacks. Leveraging a Long Short-Term Memory (LSTM) based machine learning model, the system identifies characteristics unique to phishing websites—such as specific URL patterns, domain properties, and site content features. Built with a FastAPI backend and a simple HTML, CSS, and JavaScript frontend, this detection system aims to provide real-time alerts, warning users before they engage with potentially malicious sites. Trained on a publicly available dataset, the model is designed for continuous improvement, adapting to new phishing techniques as they emerge, ensuring robust and up-to-date protection against evolving cyber threats.

Keywords: *LSTM, FastAPI, Javascript, machine learning, feature extraction, phishing detection, etc.*

TABLE OF CONTENTS

SUPERVISOR’S RECOMMENDATION.....	ii
LETTER OF APPROVAL	iii
ACKNOWLEDGMENT.....	iv
ABSTRACT	v
LIST OF FIGURES	viii
LIST OF TABLES	ix
LIST OF ABBREVIATIONS	x
CHAPTER 1: INTRODUCTION	1
1.1. Introduction.....	1
1.2. Problem Statement	1
1.3. Objectives	2
1.4. Scope and Limitation	2
1.5. Development Methodology	3
1.6. Report Organization	4
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW.....	5
2.1. Background Study	5
2.1.1 Deep learning.....	5
2.2. Literature Review	6
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN.....	7
3.1. System Analysis.....	7
3.1.1. Requirement Analysis.....	7
i. Functional Requirements.....	7
ii. Non Functional Requirements.....	8
3.1.2. Feasibility Analysis	8
i. Technical	8
ii. Operational	8
iii. Economic	8
iv. Schedule.....	9
3.1.3. Object Modelling.....	9
3.1.3.1. Object modelling using class and object diagram	9

3.1.4. Dynamic Modelling	10
3.1.4.1. Sequence Diagram	10
3.1.4.2. State Diagram	11
3.1.5. Process modelling using activity diagram.....	12
3.2. System Design	14
3.2.1. Refinement of Classes and Object.....	14
3.2.2. Deployment Diagram	15
3.3. Algorithms Analysis	17
3.3.1. LSTM:	17
3.3.2. CNN-LSTM:	19
3.3.3. Bidirectional LSTM Model:	21
CHAPTER 4: IMPLEMENTATION AND TESTING	24
4.1. Implementation	24
4.1.1. Tools Used	24
4.1.2. Implementation Details of Modules	26
4.1.2.1. Frontend Implementation.....	26
4.1.2.2. Backend Implementation	29
4.1.2.3. Model Implementation	31
4.2. Testing	35
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATION.....	38
5.1. Conclusion	38
5.2. Lesson learnt/ Outcome	38
5.3. Future Recommendation	39
REFERENCES.....	40
APPENDIX	41

LIST OF FIGURES

Figure 1: V Model.....	3
Figure 2: Use-Case Diagram.....	7
Figure 3: Gantt Chart of Phishing Link Detection.....	9
Figure 4: Class Diagram of Phishing Link Detection System	9
Figure 5: Sequence Diagram of Phishing Link Detection System	10
Figure 6: State Diagram of Phishing Link Detection System.....	11
Figure 7: Activity Diagram of Phishing Link Detection System.....	12
Figure 8: Refined class diagram of Phishing Link Detection System	14
Figure 9: Deployment Diagram of Phishing Link Detection System	15
Figure 10: LSTM architecture of Phishing Link Detection System	17
Figure 11: LSTM Learning curve of Phishing Link Detection System.....	18
Figure 12: Architecture of CNN algorithm of Phishing Link Detection System	19
Figure 13: CNN LSTM Learning Curve of Phishing Link Detection System	20
Figure 14: Bidirectional LSTM architecture of Phishing Link Detection System	21
Figure 15: Bidirectional Learning curve of Phishing Link Detection System.....	22
Figure 16: Frontend Folder Implementation Structure of Phishing Link Detection System ...	26
Figure 17: Figure: Frontend Functionalities of Phishing Link Detection System	26
Figure 18: Implementation of predictURL of Phishing Link Detection System	27
Figure 19: Implementation of proceedURL of Phishing Link Detection System	28
Figure 20: Implementation of resetPage of Phishing Link Detection System.....	28
Figure 21: Backend Project Structure of Phishing Link Detection System.....	29
Figure 22: Backend Implementation of Phishing Link Detection System	30
Figure 23: Dataset Implementation of Phishing Link Detection System	31
Figure 24: Data Preprocessing Implementation of Phishing Link Detection System	32
Figure 25: Model Implementation Of Phishing Link Detection System	33
Figure 26: Training Implementation of Phishing Link Detection System.....	34

LIST OF TABLES

Table 1: Unit Testing	35
Table 2: System Testing.....	36

LIST OF ABBREVIATIONS

AI:	Artificial Intelligence
API:	Application Programming Interface
CNN:	Convolutional Neural Network
CSS:	Cascading Style Sheets
HDF5:	Hierarchical Data Format Version 5
HTML:	Hyper Text Markup Language
JS:	JavaScript
LSTM:	Long Short-Term Memory
MI:	Machine Learning
REST:	Representational State Transfer
RNN:	Recurrent Neural Network
UI:	User Interface
V-Model:	Verification and Validation Model
XML:	Extensible Markup Language

CHAPTER 1: INTRODUCTION

1.1. Introduction

Phishing websites have become a primary method for cybercriminals to steal sensitive information from unsuspecting users. These fraudulent sites are designed to closely mimic the appearance and functionality of legitimate websites, often convincing users to enter personal details, login credentials, or financial information. As a result, phishing attacks can lead to severe consequences, including identity theft, financial losses, and data breaches.

This project seeks to address this critical issue by developing a phishing site detection system that helps safeguard users from falling victim to these deceptive attacks. The primary objective is to build a machine learning model that can accurately detect phishing websites and alert users before they inadvertently provide sensitive information to malicious actors.

To achieve this, the project will utilize publicly available datasets containing features related to phishing websites, such as URL patterns, domain information, page content, and other technical aspects that distinguish phishing sites from legitimate ones. By training a machine learning model on this dataset, the system will learn to identify patterns and anomalies that are indicative of phishing attempts.

Once trained, the system will provide warnings to users attempting to visit phishing sites, offering an essential layer of protection against one of the most prevalent forms of cyber threats.

1.2. Problem Statement

Phishing remains one of the most prominent and persistent cybersecurity threats that continues to target individuals and organizations globally. This malicious activity involves fraudulent attempts to acquire sensitive information, such as usernames, passwords, and financial details, by disguising as trustworthy entities. Phishing attacks can lead to significant financial losses, identity theft, and reputational damage for both individuals and businesses.

Despite ongoing efforts to combat phishing, current cybersecurity measures often prove to be reactive rather than proactive. Traditional defenses, such as email filtering and spam detection, while useful, are increasingly less effective against the ever-evolving tactics used

by cybercriminals. Attackers constantly adapt their methods, utilizing sophisticated techniques like social engineering, advanced spoofing, and highly personalized phishing emails that are harder to distinguish from legitimate communications.

As a result, many organizations and individuals remain vulnerable to these increasingly sophisticated attacks. The reactive nature of current cybersecurity strategies, which typically respond to phishing incidents after they occur, leaves critical gaps that attackers can exploit.

1.3. Objectives

- Develop an advanced phishing link detection system using machine learning.
- Enhance online security by real-time identification and blocking of malicious URLs.
- Promote user awareness and education about phishing risks.
- Optimize performance for fast, efficient, and reliable real-time detection.
- Provide a user-friendly interface for easy integration and use of the system.

1.4. Scope and Limitation

This project focuses on developing an end-to-end machine learning application for phishing URL detection, offering a user-friendly interface for submitting URLs and receiving real-time phishing detection results. The system aims to enhance online security by accurately identifying and blocking phishing attempts, thereby protecting users from potential financial losses and identity theft. Additionally, it is designed to integrate with existing cyber security frameworks, such as firewalls and antivirus software, to provide comprehensive protection. The detection system will be continuously updated and refined to adapt to evolving phishing tactics and emerging cyber threats.

However, the system does have certain limitations. It is specifically designed to detect phishing URLs and may not be capable of identifying other forms of cyber threats, such as malware or ransom ware. Furthermore, the system might struggle to accurately detect new or highly sophisticated phishing techniques that are not well-represented in the training data. There is also a potential for false positives, where legitimate URLs are incorrectly flagged as phishing attempts, and false negatives, where phishing URLs go undetected. These limitations can affect the overall accuracy and reliability of the system, requiring ongoing refinement to minimize their impact.

1.5. Development Methodology

The V-model development methodology can be effectively applied to the creation of a phishing link detection system, ensuring a structured and rigorous development process. This model emphasizes a sequential, validation-driven approach where each development phase is paired with a corresponding testing phase, enabling early identification and resolution of potential issues. The system's requirements, including the need for accurate phishing detection, scalability, and user-friendliness, are gathered and analyzed in the initial stages. These requirements directly influence the design of both the LSTM-based detection algorithm and the architecture of the FastAPI backend. Subsequent phases involve the detailed design of components like data preprocessing, model training, API integration, and frontend development using HTML, CSS, and JS. Each phase is validated through rigorous testing, such as unit testing of model components, integration testing of APIs, and system testing to evaluate end-to-end functionality. This structured approach ensures that the detection system meets performance, security, and usability criteria before deployment, while fostering a clear alignment between development goals and quality assurance activities.

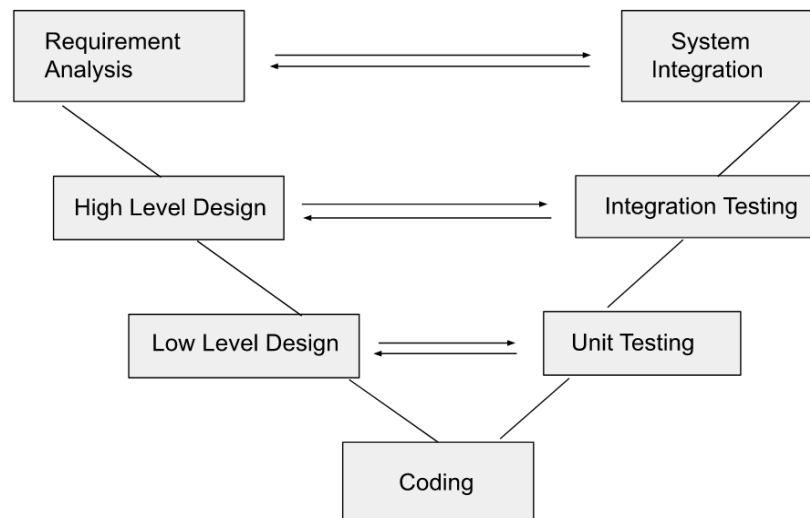


Figure 1: V Model

1.6. Report Organization

Chapter 1

This chapter introduces the core problem of phishing attacks and their implications for individuals and organizations. It outlines the project's objective to develop a phishing link detection system leveraging machine learning techniques. The chapter also presents the problem statement, objectives, and scope, providing a comprehensive overview of the project's goals, limitations, and the chosen development methodology.

Chapter 2

This chapter explores the foundational concepts related to the project, such as deep learning and Long Short-Term Memory (LSTM) networks. It provides a literature review of existing phishing detection solutions, analyzing their methodologies and technologies, including comparisons between traditional machine learning and modern deep learning techniques.

Chapter 3

In this chapter, the system's architecture and design are described in detail. It includes system requirements analysis (functional and non-functional), feasibility analysis (technical, operational, and economic), and various diagrams such as use-case, class, sequence, and activity diagrams. These provide a clear understanding of the system's structure and workflow.

Chapter 4

This chapter discusses the implementation details of the phishing link detection system, including the tools and technologies used, the system's components, and the methodologies applied to develop and deploy the system. It also explains the testing process, providing test cases and results for unit and system testing to ensure the system functions as intended.

Chapter 5

The final chapter summarizes the project's achievements and limitations. It also provides recommendations for future improvements and enhancements to make the system more robust, accurate, and adaptable to emerging phishing techniques. This section ensures the project's potential for continued development and optimization.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1. Background Study

This section consists of background study on terminologies related to our project and a review of existing solutions.

2.1.1 Deep learning

It is a field of machine learning inspired by the working of the human brain. It is an artificial intelligence method intended to mimic humans and their actions based on human brain functions for effective decision making.

Artificial Neural Networks are the heart of Deep learning algorithms. Deep learning architectures such as Deep Neural Networks, Convolutional Neural Networks, Recurrent Neural Networks are applied to the field of Computer Vision, Natural Language Processing, Speech recognition, and many domains.

2.1.2 Long Short-Term Memory

LSTM (Long Short-Term Memory) [1] is a type of recurrent neural network (RNN) [2] architecture that is particularly well-suited for analyzing and processing sequential data, such as text, time series, and natural language. Unlike traditional RNNs, which can suffer from the vanishing gradient problem, LSTM networks are designed to maintain long-term dependencies in the data, allowing them to effectively capture contextual information and learn patterns over extended sequences.

In the context of this phishing link detection system, the LSTM model will be trained to analyze the textual content and structure of suspected phishing websites, as well as the sequence of user interactions and navigation patterns. By leveraging the LSTM's ability to retain and process long-term dependencies, the model can identify subtle anomalies and suspicious characteristics that may indicate a phishing attempt, even in complex or obfuscated websites. This powerful sequence-to-sequence modeling capability makes LSTM a compelling choice for building a robust and adaptive phishing detection system that can stay ahead of evolving phishing tactics and techniques.

2.2. Literature Review

Existing phishing detection systems have explored various approaches to combat the growing threat of fraudulent websites. PhishShield [1] employs a Random Forest algorithm to detect phishing URLs, extracting a range of features from the URLs such as domain age, URL length, and presence of special characters. By training the Random Forest model on these URL-based characteristics, the system is able to learn patterns that distinguish phishing URLs from legitimate ones. On the other hand, EasyDMARC [2] utilizes a Long Short-Term Memory (LSTM) neural network to analyze the sequential patterns present in URLs. Leveraging the LSTM's ability to capture long-term dependencies in sequential data, the system can identify subtle anomalies and suspicious characteristics indicative of phishing attempts. These two contrasting approaches, one using traditional machine learning and the other employing deep learning techniques, demonstrate the diverse methodologies being explored to enhance the detection and prevention of phishing websites.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1. System Analysis

The project will be explained using dataflow diagrams, flowcharts, use-case diagrams, connection and entity diagrams, and so on.

3.1.1. Requirement Analysis

The Phishing Link detection System requirement analysis entails identifying and defining the features, functionalities, and requirements necessary for the website to run effectively and efficiently.

i. Functional Requirements

- The system shall allow users to enter url for analysis.
- The system shall analyze url that is entered by user in real time to detect phishing links. .
- The system shall view result and warn the users about the sites.

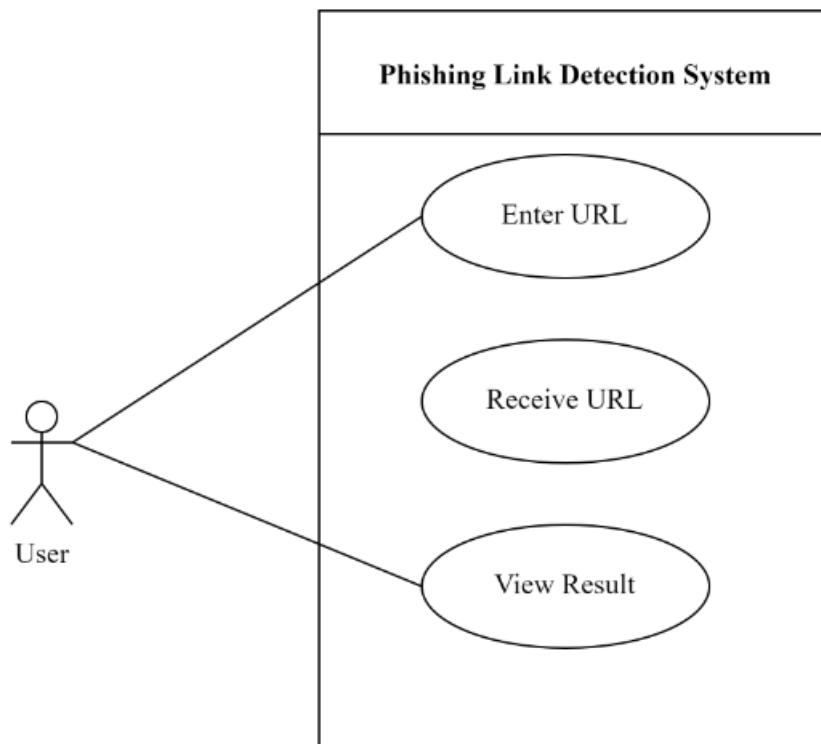


Figure 2: Use-Case Diagram of Phishing Link Detection System

ii. Non Functional Requirements

- The system must have an intuitive UI.
- The system must be secured.
- The system must have a good response.

3.1.2. Feasibility Analysis

i. Technical

Our project designs the UI using HTML, CSS and JavaScript for smooth user experience. We have a backend on FastAPI that loads the model and exposes prediction endpoint which returns if the URL is phishing or not along with the model's prediction percentage. We use hardware, software with the latest version and existing technologies.

ii. Operational

The operation feasibility determines if a system can be successfully deployed and operated reliably. Because the suggested system is web-based, it is platform independent. It is simple to access through a variety of platforms. The simple user interface will be simple to grasp and use.

iii. Economic

Most materials are freely available, and most software can be utilized complies with fair use laws and uses open-source technologies. Thus, developing the project had no associated costs. The proposed system is therefore economically feasible. However, as we are building a machine learning model, we are restricted by hardware limitations on how much training we can do. We can use dedicated cloud solutions for training our model but it would add costs.

iv. Schedule

Activity	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Requirement and gathering analysis												
Design												
Coding and Implementation												
Testing												
Deployment												
Project Documentation												

Figure 3: Gantt Chart of Phishing Link Detection

3.1.3. Object Modelling

3.1.3.1. Object modelling using class and object diagram

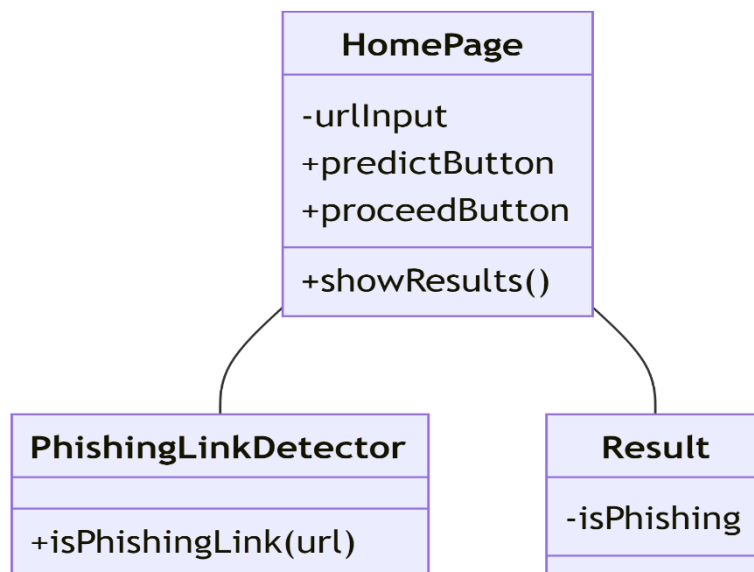


Figure 4: Class Diagram of Phishing Link Detection System

In figure 4, it shows structure of a phishing link detection system. The **HomePage** class serves as the user interface, allowing users to input a URL (`urlInput`) and interact using buttons like `predictButton` to initiate prediction and `proceedButton` to continue after results are displayed. It has a method `showResults()` to display outcomes. The **PhishingLinkDetector**

class provides the core functionality through the `isPhishingLink(url)` method, which determines if a given URL is a phishing link. The `Result` class encapsulates the outcome with an `isPhishing` attribute, indicating whether the URL is safe or malicious. The relationship reflects data flow from user input to detection and result display.

3.1.4. Dynamic Modelling

3.1.4.1. Sequence Diagram

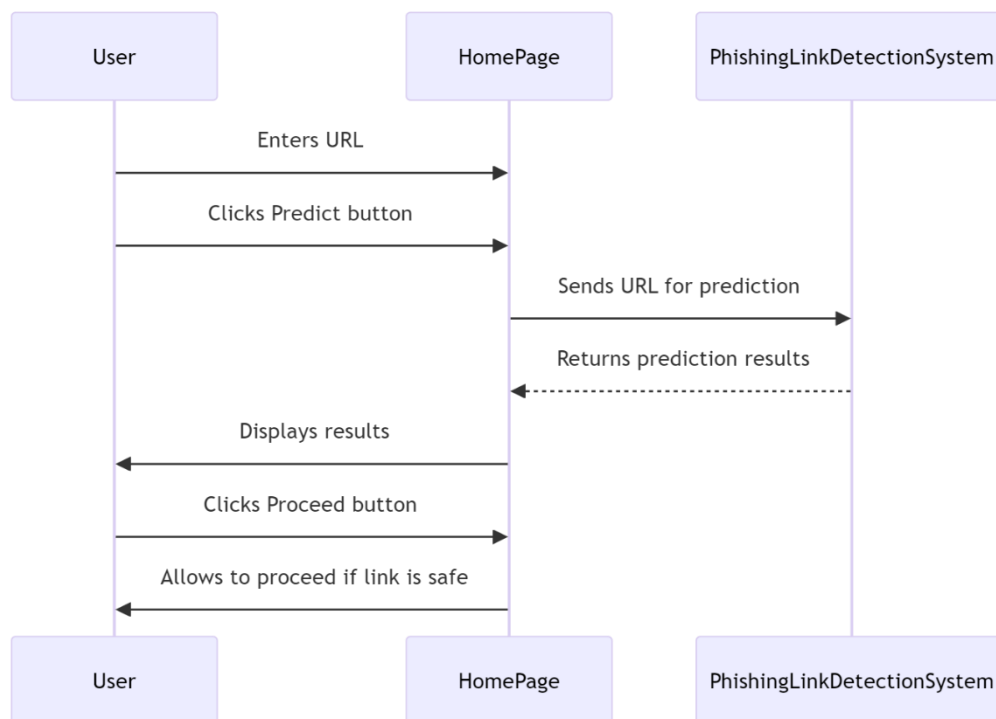


Figure 5: Sequence Diagram of Phishing Link Detection System

In figure 4, it shows the interaction flow in a phishing link detection system. A user enters a URL on the homepage and clicks the "Predict" button. The homepage sends the URL to the phishing detection system, which processes it and returns prediction results. The results are displayed to the user. If the link is safe, the user can proceed by clicking the "Proceed" button, and the system allows further actions. This ensures that unsafe links are identified and blocked before the user interacts with them.

3.1.4.2. State Diagram

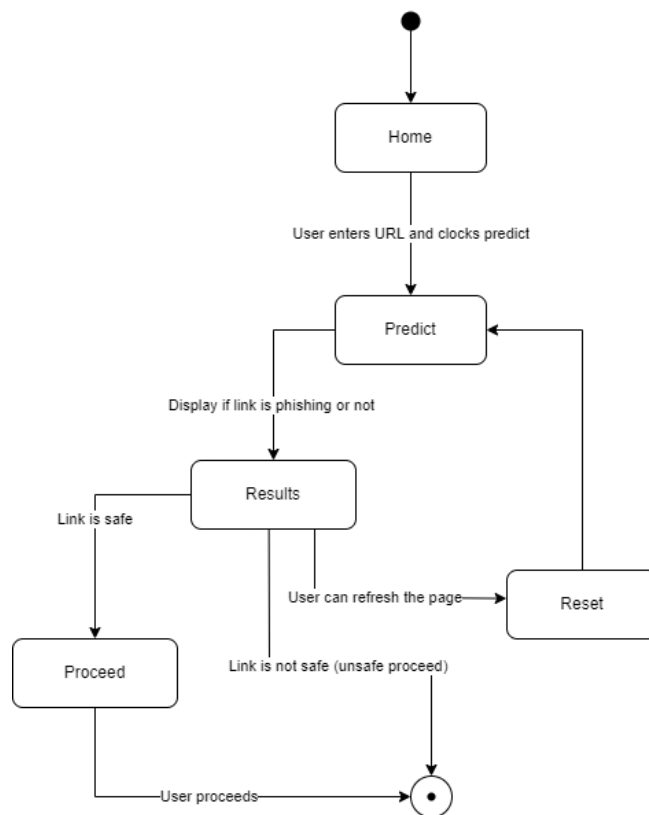


Figure 6: State Diagram of Phishing Link Detection System

In figure 5, it shows user flow for a phishing link detection system. The user begins on the "Home" page. They then input a URL into the designated box on the "Enter URL" screen. Clicking the "Predict" button initiates the analysis process. The system then displays the results, indicating whether the URL is deemed a phishing link or not on the "Show Results" screen. Only after viewing these results can the user proceed by clicking the "Proceed" button, moving them to the final stage of the process. If "Reset" is pressed the state gets reset to "Predict" resetting the flow.

3.1.5. Process modelling using activity diagram

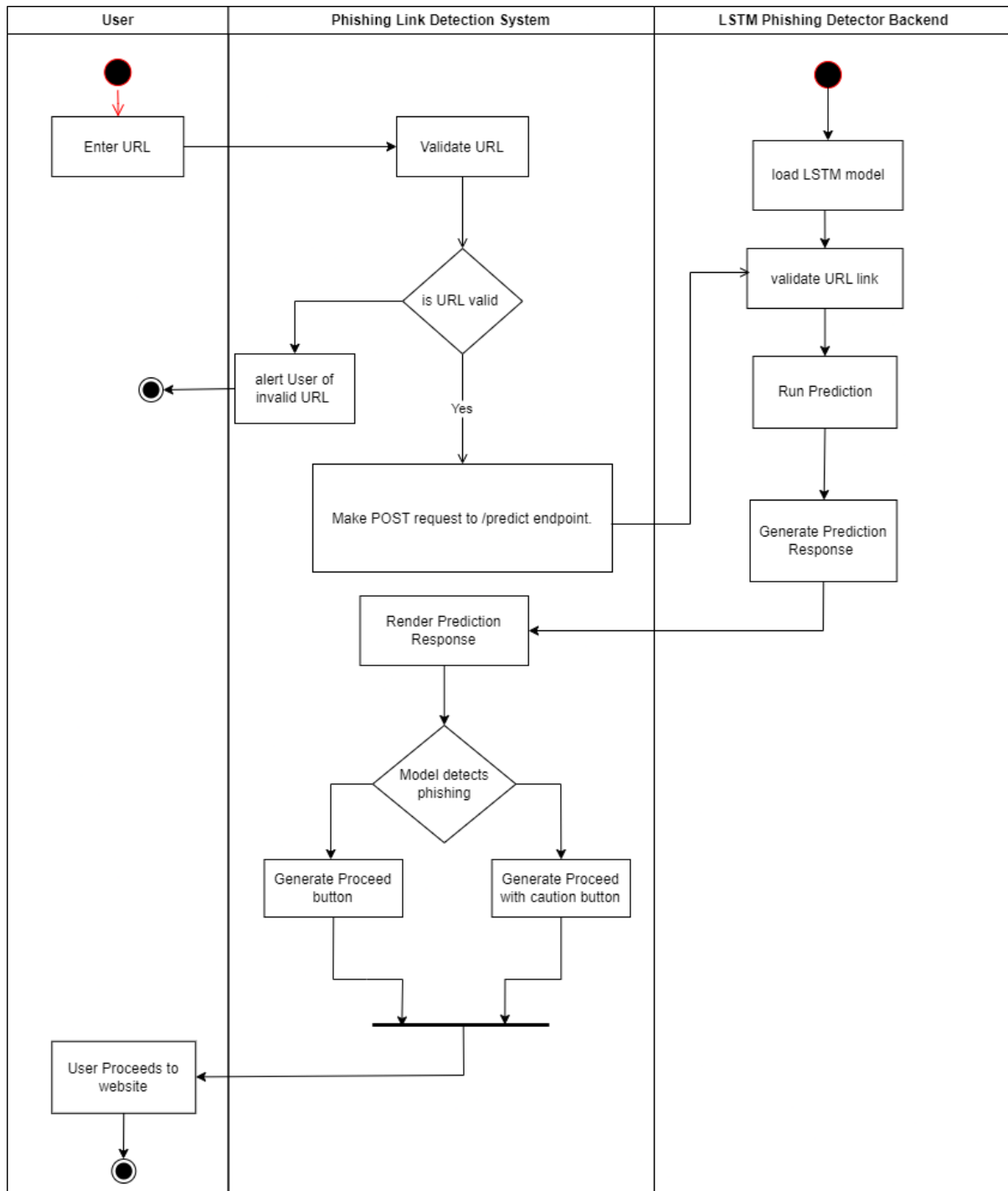


Figure 7: Activity Diagram of Phishing Link Detection System

This activity diagram represents the process flow of a phishing link detection system that interacts with users and an LSTM-based phishing detection backend. The process begins with the user entering a URL into the system. The phishing link detection system validates the URL to ensure it is formatted correctly. If the URL is invalid, the system alerts the user and

terminates the process. If the URL is valid, the system makes a POST request to the /predict endpoint of the LSTM phishing detector backend.

The backend starts by loading the pre-trained LSTM model. It then validates the URL format and runs the phishing detection prediction using the LSTM model. Once the prediction is completed, the backend generates a prediction response, which it sends back to the phishing link detection system.

Upon receiving the prediction response, the phishing detection system evaluates whether the model has flagged the URL as phishing. If the URL is classified as phishing, the system generates a "Proceed with caution" button, warning the user about the potential risk. If the URL is deemed safe, a regular "Proceed" button is generated. The user can then proceed to the website based on their choice, marking the end of the process.

This diagram highlights the interaction between the user, the phishing detection system, and the backend, focusing on validation, prediction, and user decision-making. It ensures a structured approach to detecting and managing potentially harmful URLs.

3.2. System Design

3.2.1. Refinement of Classes and Object

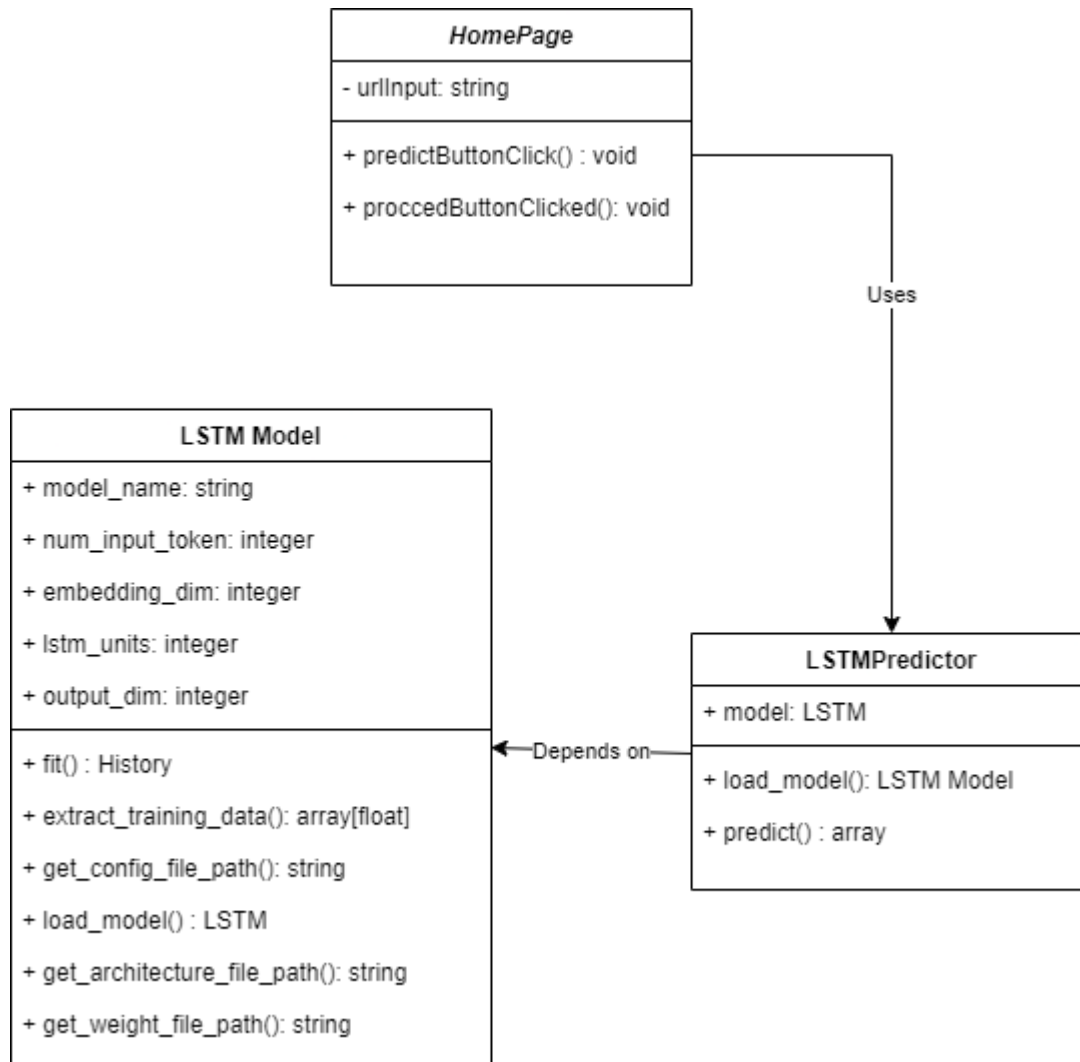


Figure 8: Refined class diagram of Phishing Link Detection System

The class diagram depicts a system with three interconnected components. The **HomePage** class represents the user interface, where users input a URL via the `urlInput` attribute. It provides two key methods: `predictButtonClick()`, which initiates the prediction process, and `proccedButtonClicked()`, which performs additional actions after the prediction. The **HomePage** class interacts with the **LSTMPredictor** class to handle predictions.

The **LSTMPredictor** class serves as the intermediate component that manages the machine learning model. It holds a reference to the **LSTM** model through its `model` attribute. This

class provides methods such as `load_model()`, which loads the trained LSTM model, and `predict()`, which performs predictions and returns the results. The `LSTMPredictor` relies on the `LSTM` class to implement its functionality.

The `LSTM` class encapsulates the details of the machine learning model. It includes attributes like `model_name` (to specify the name of the model), `num_input_token` (the number of input tokens), `embedding_dim` (dimension of word embeddings), `lstm_units` (number of LSTM units), and `output_dim` (dimension of the output). It provides methods such as `fit()`, which trains the model and returns a training history, `extract_training_data()` to extract training data, and methods like `get_config_file_path()`, `get_architecture_file_path()`, and `get_weight_file_path()` to retrieve the file paths for model configuration, architecture, and weights. Additionally, the `load_model()` method is used to load a trained LSTM model.

3.2.2. Deployment Diagram

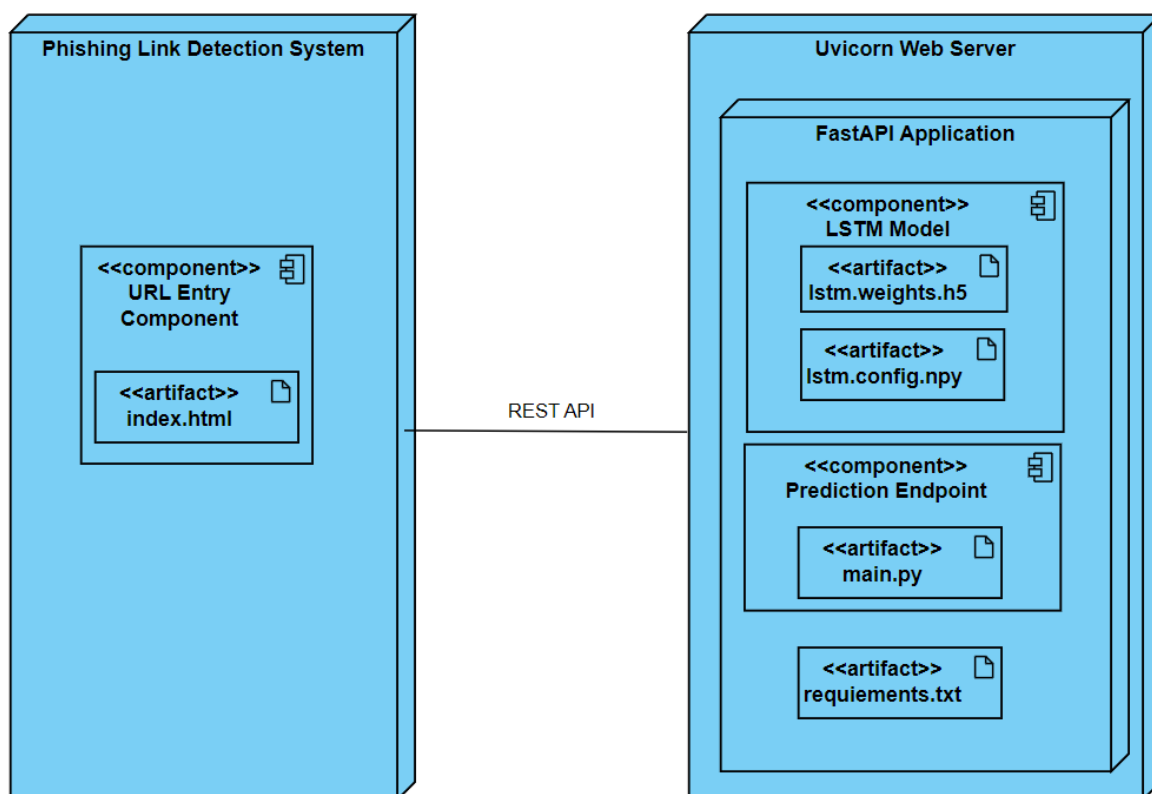


Figure 9: Deployment Diagram of Phishing Link Detection System

This deployment diagram illustrates the architecture of a phishing link detection system and

highlights the interaction between the client-side and server-side components.

On the left, the Phishing Link Detection System is represented as a standalone component with a URL Entry Component. This component handles user interactions, such as accepting a URL input for validation and prediction. It is linked to an artifact called `index.html`, which represents the front-end HTML file that facilitates the user interface.

On the right, the Uvicorn Web Server hosts the server-side application. Within this server, there is a FastAPI Application that manages the backend logic. This application contains two key components:

LSTM Model: This component is responsible for performing phishing detection. It includes two artifacts:

- `lstm.weights.h5`: The pre-trained model weights stored in an HDF5 file.
- `lstm.config.npy`: The configuration file for the LSTM model, which might include metadata such as input size, output size, or other parameters.

Prediction Endpoint: This component provides a REST API endpoint for predictions under `/predict` endpoint. It includes the artifact `main.py`, which contains the Python code that handles the FastAPI application logic, processes requests, and invokes the LSTM model. Additionally, it has a `requirements.txt` artifact that lists the dependencies required to run the application, ensuring the server environment is correctly configured.

The interaction between the client and the server is facilitated via a REST API. The front-end component (`index.html`) sends requests to the server's prediction endpoint, which processes the input, runs the LSTM model, and returns the result. This deployment diagram effectively captures the modular structure, showing the separation of concerns between the user interface, backend API, and the LSTM model, ensuring scalability and maintainability

3.3. Algorithms Analysis

There are three LSTM algorithm variations for this phishing link detection system.

3.3.1. LSTM:

LSTM, or Long Short-Term Memory, is a type of recurrent neural network (RNN) that can process and retain information over multiple time steps. LSTMs use gates to control the flow of information, allowing them to selectively retain or discard information as needed. This helps prevent the issues of gradient exploding and vanishing that occur in standard RNNs.

Our model architecture is the following:

```
NB_LSTM_CELLS = 256
NB_DENSE_CELLS = 256
def make_lstm_model(num_input_tokens):
    model = Sequential()
    model.add(LSTM(NB_LSTM_CELLS, input_shape=(None, num_input_tokens),
        return_sequences=False,
        return_state=False,
        dropout=0.2)
    model.add(Dense(NB_DENSE_CELLS))
    model.add(Dropout(0.3))
    model.add(Dense(2))
    model.add(Activation('softmax'))
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

Figure 10: LSTM architecture of Phishing Link Detection System

The model starts with an LSTM layer comprising 256 units, which processes input sequences of variable length and a specified number of features per timestep. This layer captures temporal dependencies in the data and outputs a fixed-length representation while applying dropout to reduce overfitting. Following the LSTM layer, a Dense layer with 256 units is included to further transform the output, introducing additional learning capacity for complex patterns. A Dropout layer is then added to randomly deactivate a fraction of neurons during training, enhancing generalization. The final Dense layer reduces the output to two units, corresponding to the target classes, and applies a softmax activation function to generate class probabilities. The model is compiled using the Adam optimizer, categorical cross-entropy loss for multi-class classification, and accuracy as the evaluation metric, making it effective for tasks that require learning temporal patterns.

Model accuracy metrics:

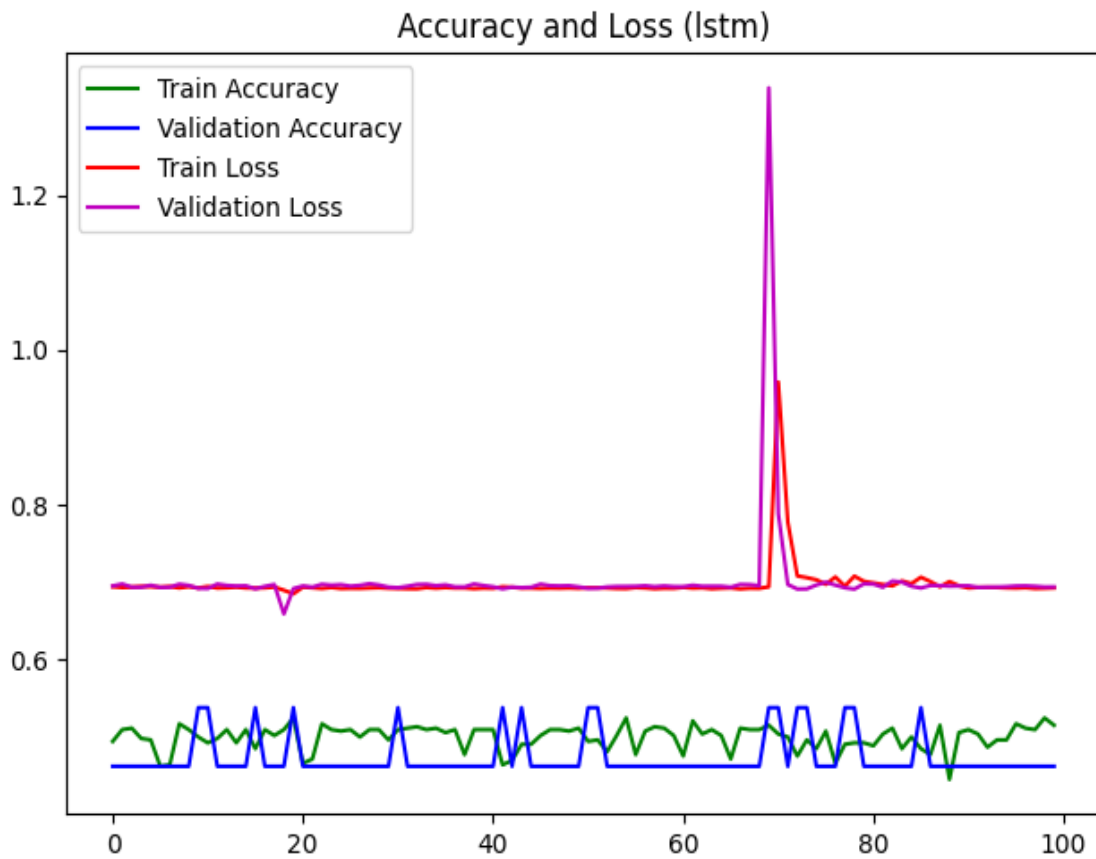


Figure 11: LSTM Learning curve of Phishing Link Detection System

This figure depicts the training and validation performance of an LSTM model over 100 epochs, showing accuracy and loss for both datasets. The green line represents training accuracy, while the blue line indicates validation accuracy. Both lines show minimal improvement throughout the training process and remain low, with fluctuations suggesting that the model struggles to learn effectively or generalize well. The red line, representing training loss, decreases slightly over time but stays relatively stable, indicating slow or insufficient optimization during training. Meanwhile, the magenta line, representing validation loss, exhibits an unusual spike around the 60th epoch before stabilizing again. This spike could indicate an anomaly or instability during training, such as a learning rate issue or a sudden overfitting phase. Overall, the figure suggests that the LSTM model is underperforming, with neither accuracy nor loss metrics demonstrating significant improvement or stability.

3.3.2. CNN-LSTM:

CNN-LSTM is a deep learning architecture that combines Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTMs) to process sequential data.

The model architecture of the system:

```
NB_LSTM_CELLS = 256
NB_DENSE_CELLS = 256
EMBEDDING_SIZE = 100

def make_cnn_lstm_model(num_input_tokens, max_len):
    model = Sequential()
    model.add(Embedding(input_dim=num_input_tokens, input_length=max_len, output_dim=EMBEDDING_SIZE))
    model.add(SpatialDropout1D(0.2))
    model.add(Conv1D(filters=256, kernel_size=5, padding='same', activation='relu'))
    model.add(MaxPooling1D(pool_size=4))
    model.add(LSTM(NB_LSTM_CELLS))
    model.add(Dense(units=2, activation='softmax'))
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

Figure 12: Architecture of CNN algorithm of Phishing Link Detection System

The model is initialized with specific hyperparameters: 256 LSTM cells, 256 dense cells, and an embedding dimension of 100. The function 'make_cnn_lstm_model' takes two parameters: the vocabulary size (num_input_tokens) and the maximum sequence length (max_len). The model begins with an Embedding layer that transforms discrete tokens into 100-dimensional dense vectors, followed by a SpatialDropout1D layer with a rate of 0.2 for regularization. The key feature of this architecture is the combination of a Convolutional Neural Network (CNN) layer with 256 filters and a kernel size of 5, which extracts local patterns from the embedded sequence, with an LSTM layer that captures long-term dependencies. Between these, a MaxPooling1D layer with a pool size of 4 reduces the sequence length and helps extract the most important features. The LSTM layer processes these CNN-extracted features with 256 units, capturing sequential patterns in the data. Finally, a Dense layer with softmax activation produces binary classification probabilities. The model is compiled using the Adam optimizer and categorical crossentropy loss function.

Model accuracy metric

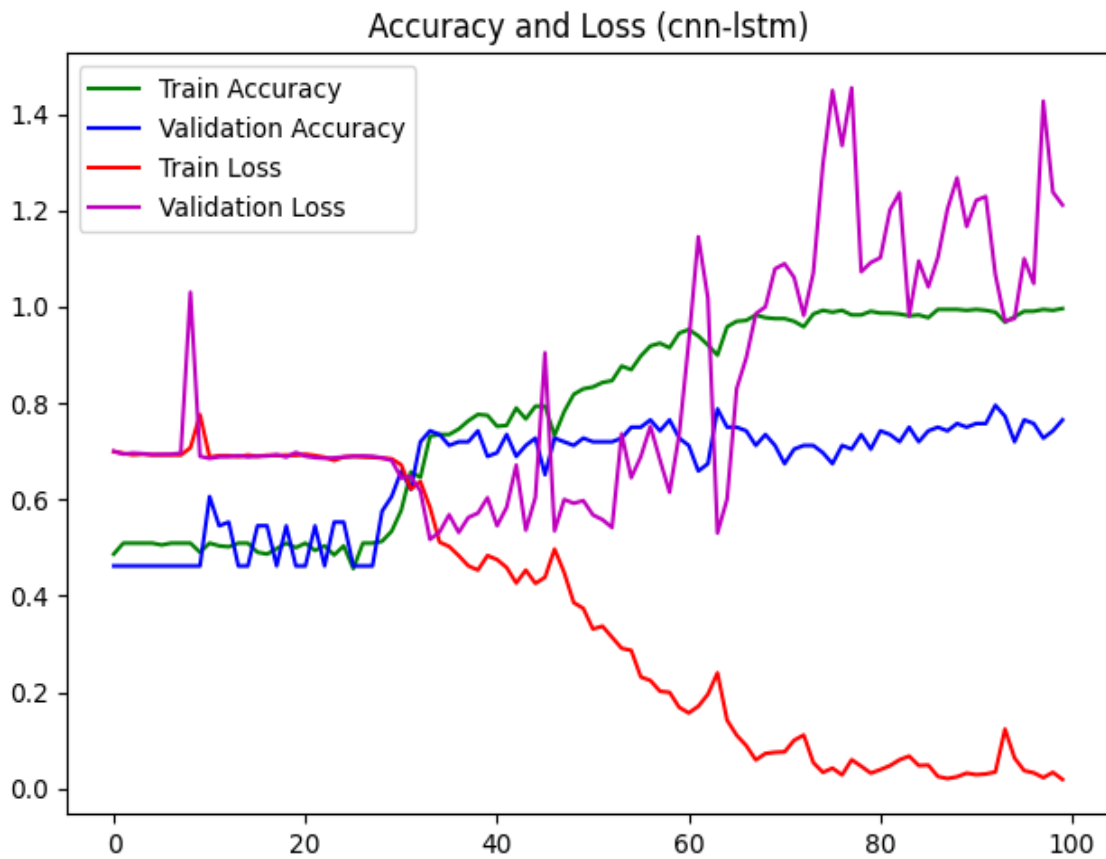


Figure 13: CNN LSTM Learning Curve of Phishing Link Detection System

This figure shows the training and validation performance of CNN-LSTM model over 100 epochs, showing accuracy and loss for both datasets. The green line (train accuracy) represents training accuracy, while the blue line (validation accuracy) indicates validation accuracy. Both lines show minimal improvement throughout the training process and remain low, with fluctuations suggesting that the model struggles to learn effectively or generalize well. The red line (train loss), representing training loss, decreases slightly over time but stays relatively stable, indicating slow or insufficient optimization during training. Meanwhile, the magenta line (validation loss), representing validation loss, exhibits an unusual spike around the 60th epoch before stabilizing again. This spike could indicate an anomaly or instability during training, such as a learning rate issue or a sudden overfitting phase. Overall, the figure suggests that the CNN-LSTM model is underperforming, with neither accuracy nor loss metrics demonstrating significant improvement or stability.

3.3.3. Bidirectional LSTM Model:

Bidirectional Long Short-Term Memory (BiLSTM) is a type of recurrent neural network (RNN) that processes sequential data in both forward and backward directions. BiLSTM works by presenting each training sequence forward and backward to two independent LSTM networks. It couples both LSTM networks to the same output layer concatenating the results from both forward and backward LSTM at each time step

```
NB_LSTM_CELLS = 256
NB_DENSE_CELLS = 256
EMBEDDING_SIZE = 281
def make_bidirectional_lstm_model(
    num_input_tokens,
    embedding_dim=EMBEDDING_SIZE,
    lstm_units=NB_LSTM_CELLS,
    output_dim=2
):
    model = Sequential([
        Embedding(
            input_dim=num_input_tokens,
            output_dim=embedding_dim,
            input_length=None # Allow variable length input
        ),
        SpatialDropout1D(0.2),
        Bidirectional(LSTM(
            units=lstm_units,
            dropout=0.2,
            recurrent_dropout=0.2
        )),
        Dense(output_dim, activation='softmax')
    ])

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

Figure 14: Bidirectional LSTM architecture of Phishing Link Detection System

The model begins with an Embedding layer that converts input tokens into dense vector representations of a specified dimension, allowing for variable-length input sequences. This embedding process transforms discrete tokens into continuous-valued vectors that capture semantic relationships between tokens. Following the Embedding layer, a SpatialDropout1D

layer is applied to reduce overfitting by randomly dropping a fraction of the embedding dimensions during training. The core of the model is a Bidirectional LSTM layer with 256 units, where sequences are processed in both forward and backward directions, enhancing the model's ability to capture context and dependencies from both ends of the input sequence. This layer incorporates dropout and recurrent dropout to prevent overfitting and ensure robustness. The final Dense layer maps the output of the LSTM to a two-dimensional output space, using a softmax activation function to produce probabilities for the target classes. The model is compiled with the Adam optimizer, categorical cross-entropy loss, and accuracy as the evaluation metric, making it suitable for multi-class classification tasks.

Model analysis metrics

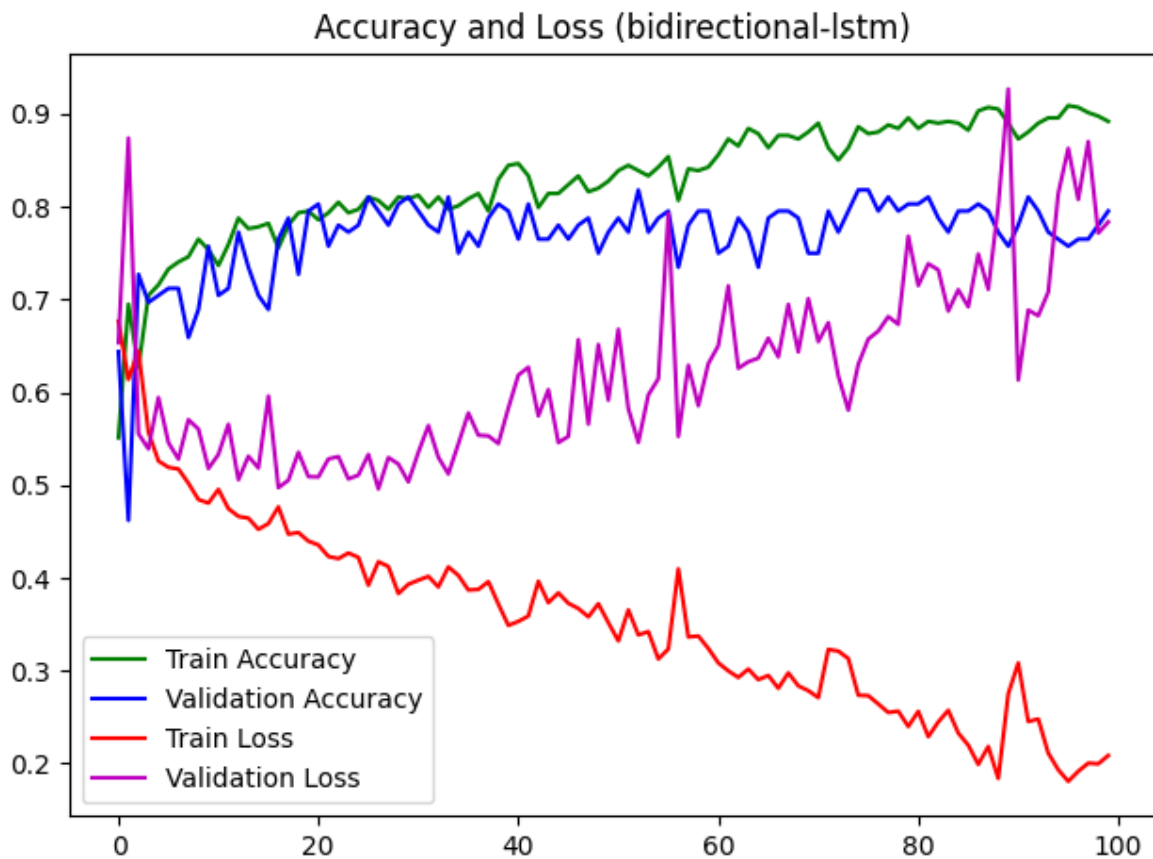


Figure 15: Bidirectional Learning curve of Phishing Link Detection System

This figure illustrates the training and validation performance of a bidirectional LSTM model over 100 epochs, showcasing accuracy and loss for both datasets. The green line represents training accuracy, which steadily improves and reaches a high value, indicating that the

model is learning well on the training data. The blue line represents validation accuracy, which also improves but fluctuates more, reflecting variability in the model's generalization to unseen data. The red line shows training loss, which decreases consistently, demonstrating that the model minimizes error effectively during training. Meanwhile, the magenta line, representing validation loss, initially decreases but later fluctuates significantly, suggesting potential overfitting. This behavior occurs because, while the model continues to fit the training data well, its performance on unseen validation data becomes less stable. The divergence between training and validation metrics in the later epochs indicates that the model might be overfitting, learning patterns specific to the training data rather than generalizable features. Overall, while the bidirectional LSTM demonstrates strong learning on the training data, its validation performance highlights opportunities for refinement.

From the three algorithmic variation, we see bidirectional LSTM shows the best outcome. Hence we decided to move forward with implementing bidirectional LSTM for our phishing link detection model.

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1. Implementation

Implementation refers to the process of translating a concept or plan into a tangible reality by executing the necessary actions and steps. It involves putting the ideas, designs, and strategies into practice to create a functioning product, system, or solution. In the context of Phishing Link Detection System, implementation refers to the actual development and deployment of the system based on the defined requirements and specifications.

4.1.1. Tools Used

• PYTHON

Python is a popular, high-level programming language that is widely utilized in diverse applications like scientific computing, web development, data analysis, artificial intelligence, and more. It supports a wide range of object-oriented programming (OOP) concepts, including encapsulation, inheritance, and polymorphism. Python also boasts a rich collection of third-party frameworks and standard libraries for various applications, making it a popular choice for developers and organizations.

• HTML

HTML, which stands for Hypertext Markup Language, is a markup language that is primarily used to create web pages and web applications. HTML is used to create the simple college website in this project.

• CSS

Cascading Style Sheets (CSS) is a style sheet language that is utilized to describe the presentation of a web document written in HTML or XML. It is used to style the website which is created with HTML in this project.

• Visual Studio Code

This is our code editor where we have written our all of codes. This tool is very user friendly and have lots of extensions which helps for making the coding process more efficient.

• Snipping Tool

This is a pre-installed tool on our Windows Machine which helps to take the screenshots of all required figures, documents and user interfaces.

- **Microsoft Word**

This tool is used to do all the documentation of our project from the scratch to the very end.

- **Microsoft PowerPoint**

This tool is used to make the PowerPoint slides to do presentation of our project.

- **FastAPI (Backend Framework)**

Used for developing the backend RESTful API to handle URL submissions, interact with the machine learning model, and return prediction results to the frontend.

- **TensorFlow/Keras (Machine Learning Libraries)**

Used for building and training the LSTM model for phishing detection.

- **Scikit-learn (Machine Learning Libraries)**

Used for preprocessing data, feature extraction, and evaluation of traditional machine learning tasks.

- **Git and GitHub**

Used for version control and collaborative development, ensuring code changes are tracked and managed effectively.

4.1.2. Implementation Details of Modules

4.1.2.1. Frontend Implementation

For frontend implementation HTML, CSS and JS is used for web UI. The folder implementation structure is as:



Figure 16: Frontend Folder Implementation Structure of Phishing Link Detection System

The assets includes all the images, css that is use for the frontend UI. The systems frontend app's core logic resides in index.html under the `<script>` tag.

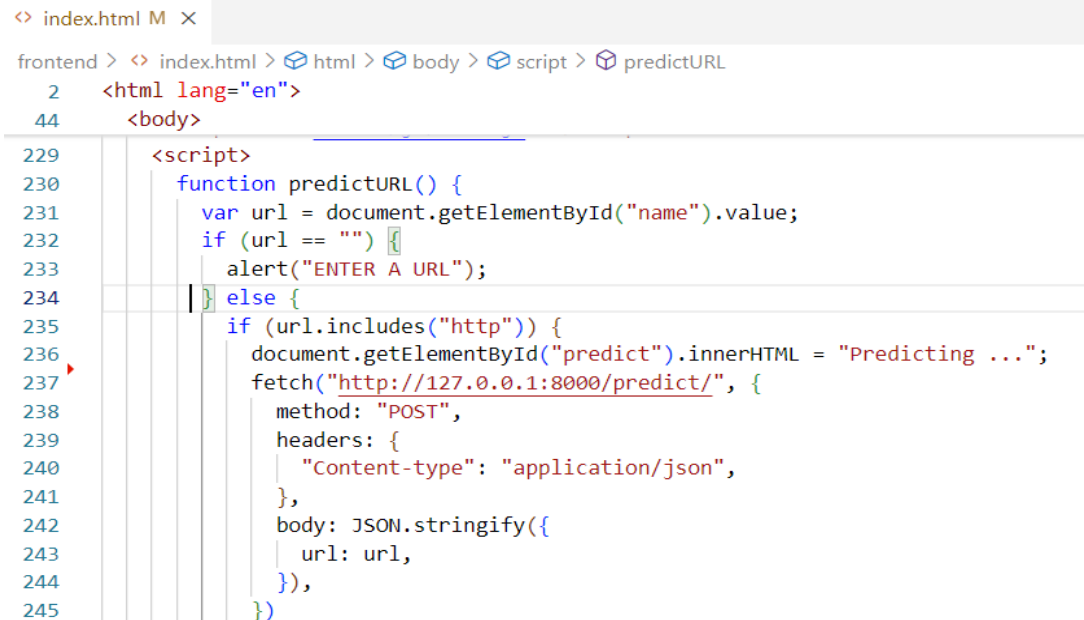
```
<script>
  function predictURL() { ...
  function proceedURL() { ...
  function resetPage() { ...
</script>
```

Figure 17: Figure: Frontend Functionalities of Phishing Link Detection System

The system has three functionalities:

1. **predictURL():**

This is responsible for validating the URL, sending the payload into the backend endpoint and dynamically updating frontend components upon getting response from backend.



```
<> index.html M X
frontend > <> index.html > html > body > script > predictURL
2   <html lang="en">
44  <body>
229 <script>
230   function predictURL() {
231     var url = document.getElementById("name").value;
232     if (url == "") {
233       alert("ENTER A URL");
234     } else {
235       if (url.includes("http")) {
236         document.getElementById("predict").innerHTML = "Predicting ...";
237         fetch("http://127.0.0.1:8000/predict/", {
238           method: "POST",
239           headers: {
240             "Content-type": "application/json",
241           },
242           body: JSON.stringify({
243             url: url,
244           }),
245         });
246       }
247     }
248   }
249 }
```

Figure 18: Implementation of predictURL of Phishing Link Detection System

Currently, the frontend url makes request to an endpoint /predict on local host. Once the data is received from backend, the system updates html classes based on responses to show if the link is phishing or not.

2. **proceedURL():**

This functionality is responsible for taking user to their desire website after phishing detection. If the model predicts the website user is trying to visit as phishing, the system shows a warning button indicating to proceed to website with caution.

```
function proceedURL() {  
    var url = document.getElementById("name").value;  
    console.log(url);  
    if (url) {  
        window.location.href = url;  
    } else {  
        alert("Please enter a valid URL.");  
    }  
}
```

Figure 19: Implementation of proceedURL of Phishing Link Detection System

3. resetPage():

This reloads the application essentially allowing users to start from the beginning

```
function resetPage() {  
    location.reload();  
}
```

Figure 20: Implementation of resetPage of Phishing Link Detection System

4.1.2.2.Backend Implementation

The project structure of backend implementation is as follows:

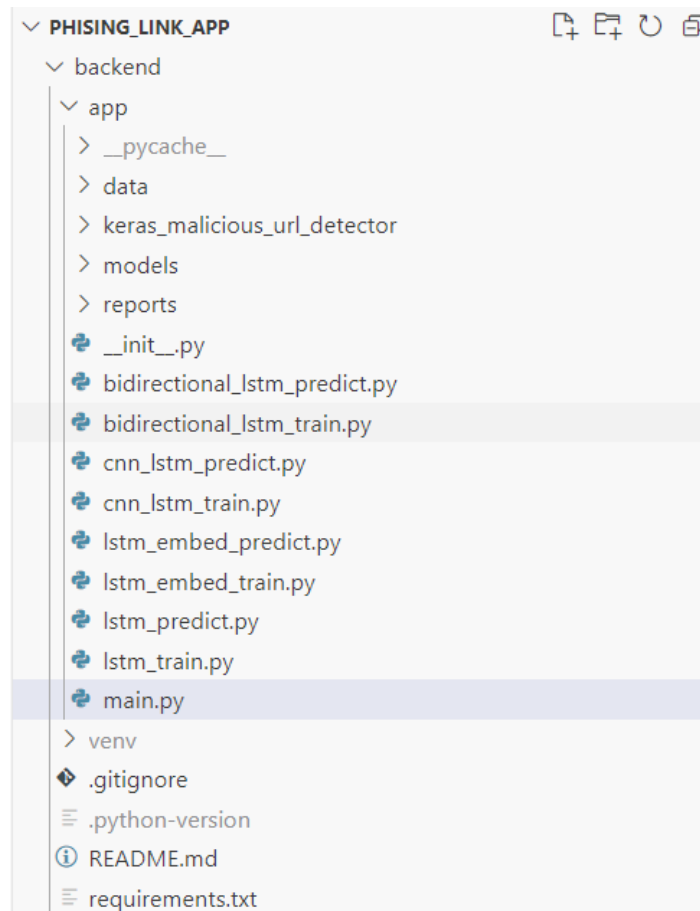


Figure 21: Backend Project Structure of Phishing Link Detection System

1. App: This folder contains the core backend. It includes following modules:
 - a. Keras malicious url detector: Includes the model building logic.
 - b. Data: URL dataset for training model.
 - c. Models: Stores the weight, configuration and architecture of trained models.
 - d. Reports: Stores the accarcy and loss reports.
 - e. Additional python files that are abstraction to train various models and prediction of the models to test each model's accuracy.
2. venv: A virtual environment built for this project where the packages needed for the project are installed.
3. .gitingore: The entire project is tracked with version control GIT, the .gitignore file is used to keep things not to push into the version control system
4. requirements.txt: File that contains a list of packages or libraries needed to work on a

project that can all be installed with the file.

The systems backend is a FastAPI based application and currently, it has the endpoint exposed under main.py

```
backend > app > main.py > ...
1
2 from keras_malicious_url_detector.library.bidirectional_lstm import BidirectionalLstmEmbedPredictor
3 from fastapi import FastAPI
4 from pydantic import BaseModel, HttpUrl
5 from fastapi.middleware.cors import CORSMiddleware
6
7 class URLData(BaseModel):
8     url: HttpUrl
9
10 app = FastAPI(title="Phishing Link Detection Backend")
11
12 app.add_middleware(
13     CORSMiddleware,
14     allow_origins=["*"],
15     allow_credentials=True,
16     allow_methods=["*"],
17     allow_headers=["*"],
18 )
19
20 model_dir_path = './models'
21 predictor = BidirectionalLstmEmbedPredictor()
22 predictor.load_model(model_dir_path)
23
24 @app.post('/predict')
25 def predict_phishing(url_data: URLData):
26     url = str(url_data.url)
27     predict, predicted = predictor.predict(url)
28
29     return {
30         "url": url,
31         "is_phishing": True if predict else False,
32         "phishing_prediction": round(predicted[-1] * 100,2),
33         "non_phishing_prediction": round(predicted[0] * 100,2)
34     }
35
```

Figure 22: Backend Implementation of Phishing Link Detection System

In systems backend app, it loads the trained model while the application is being instantiated. This allows a global predictor object that can be use for prediction.

The sysem is built on one endpoint /predict that takes in URLas input payload and it uses Pydantic and its HttpURL validator that performs backend URL validation as well. The validated URL is then passed into model's prediction functionality. The prediction functionality responds with two values: predict – which is Boolean value and predicted - which is numerical value that represents the model confidence score of the given site being phishing.

4.1.2.3. Model Implementation

The model implementation of the system resides under backend in the following representation

Dataset implementation

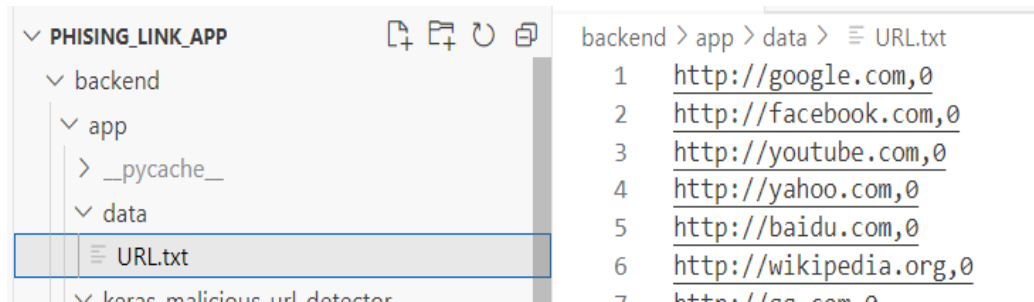


Figure 23: Dataset Implementation of Phishing Link Detection System

This contains URL dataset. There are 858 generated datasets inclusive of both phishing and non phishing websites. There is a comma separated dataset where the first value is URL and the second value indicates phishing or not (0 is non phishing, 1 represents phishing).

Data preprocessing implementation:

```
def extract_training_data(self, url_data):
    data_size = url_data.shape[0]
    X = np.zeros(shape=(data_size, self.max_url_seq_length))
    Y = np.zeros(shape=(data_size, 2))
    for i in range(data_size):
        url = url_data['text'][i]
        label = url_data['label'][i]
        for idx, c in enumerate(url):
            X[i, idx] = self.char2idx[c]
        Y[i, label] = 1

    return X, Y
```

Figure 24: Data Preprocessing Implementation of Phishing Link Detection System

The function takes a dataset containing URLs and their corresponding labels (0 for legitimate, 1 for phishing) and transforms them into two arrays: X for feature data and Y for labels. For each URL in the dataset, the function converts each character into a numerical index using a character-to-index mapping dictionary (char2idx), storing these indices in a fixed-length sequence matrix X where each row represents one URL. If a URL is shorter than the maximum length (max_url_seq_length), the remaining positions are padded with zeros. Simultaneously, it creates a one-hot encoded matrix Y for the labels, where each row contains [1,0] for legitimate URLs (label 0) or [0,1] for phishing URLs (label 1).

Model implementation

```
bidirectional_lstm.py 4 X
backend > app > keras_malicious_url_detector > library > bidirectional_lstm.py > make_bidirectional_lstm_model
6 from keras.models import Sequential
7 from keras.callbacks import ModelCheckpoint
8
9 NB_LSTM_CELLS = 256
10 NB_DENSE_CELLS = 256
11 EMBEDDING_SIZE = 281
12
13
14 def make_bidirectional_lstm_model(
15     num_input_tokens,
16     embedding_dim=EMBEDDING_SIZE,
17     lstm_units=NB_LSTM_CELLS,
18     output_dim=2
19 ):
20     model = Sequential([
21         Embedding(
22             input_dim=num_input_tokens,
23             output_dim=embedding_dim,
24             input_length=None # Allow variable length input
25         ),
26         SpatialDropout1D(0.2),
27         Bidirectional(LSTM(
28             units=lstm_units,
29             dropout=0.2,
30             recurrent_dropout=0.2
31         )),
32         Dense(output_dim, activation='softmax')
33     ])
34
35     model.compile(
36         optimizer=Adam(learning_rate=0.001),
37         loss='categorical_crossentropy',
38         metrics=['accuracy']
39     )
40
```

Figure 25: Model Implementation of Phishing Link Detection System

The model architecture is configured with specific hyperparameters: 256 LSTM cells, 256 dense layer cells, and an embedding dimension of 281, which are defined as constants at the beginning of the code.

The function 'make_bidirectional_lstm_model' takes four parameters: the number of input tokens in the vocabulary, the embedding dimension (defaulting to 281), the number of LSTM units (defaulting to 256), and the output dimension (defaulting to 2 for binary classification). The model is constructed using Keras's Sequential API, which allows for a straightforward linear stack of layers.

The first layer is an Embedding layer, which transforms discrete tokens (like characters or words) into dense vectors of fixed size. This layer takes the vocabulary size as `input_dim` and produces vectors of `embedding_dim` dimensions. It's configured to accept variable-length input sequences, making it flexible for different URL lengths. Following the embedding layer, a `SpatialDropout1D` layer with a rate of 0.2 is added to prevent overfitting by randomly dropping 20% of the entire feature maps during training.

The core of the model is the Bidirectional LSTM layer. This layer processes the sequence in both forward and backward directions, allowing the model to capture patterns that depend on both past and future context. The LSTM layer is configured with 256 units and includes both regular dropout and recurrent dropout of 0.2, which helps prevent overfitting by randomly dropping connections during training. The bidirectional wrapper effectively doubles the number of parameters in this layer, as it maintains separate weights for forward and backward passes.

The final layer is a Dense layer with softmax activation, producing output probabilities for each class (in this case, two classes for binary classification of phishing vs. legitimate URLs). The softmax activation ensures the outputs sum to 1 and can be interpreted as probabilities. The model is compiled using the Adam optimizer with a learning rate of 0.001, categorical crossentropy loss function (appropriate for multi-class classification), and accuracy as the evaluation metric.

Training Implementation

While training the model, batch size of 64 is used and train it for 65 epochs



```
batch_size = 64
epochs = 100
history = classifier.fit(text_model=text_model,
                        model_dir_path=model_dir_path,
                        url_data=url_data, batch_size=batch_size, epochs=epochs)
```

Figure 26: Training Implementation of Phishing Link Detection System

4.2. Testing

Software testing is a crucial part of software development life cycle. Software testing is the process of ensuring that the system or product works as it intend to or say meets the requirement and specification. Mostly testing should be started as early as possible in order to reduce the cost and loss of time. But testing can be done at the end of the implementation before the time of the deployment. A system or an application can never be 100% free from bugs or error. So, testing helps in separating the error with that of the successful outcome so that updating as well as bug fixing can be done at time. In this project, unit testing and system testing has been applied.

Table 1: Unit Testing

Test Id	Test Case	Test Case Description	Input Example	Expected Outcome	Actual Outcome	Remarks
1)	Data Preprocessing: Handling Missing Values	Ensure the system can handle missing or incomplete data inputs.	{“url”: “”}	Error	Error	Pass
2	URL Validation	Verify that the system correctly identifies invalid URLs.	“http://invalid-url”	Invalid URL format	Invalid URL format	Pass
4)	Model Prediction: Phishing URL	Check if the model predicts a phishing URL correctly	“http://free-win-prizes.com”	Non-Suspicious	Error	Failed
5)	Model Prediction : Legitimate URL	Check if the model predicts a legitimate URL correctly.	“https://www.google.com”	Suspicious	Suspicious	Pass

Table 2: System Testing

Test Case Id	Test Case	Test Case Description	Input Example	Expected Outcome	Actual Outcome	Remarks
1)	End-to-End Phishing URL Detection Workflow	Validate the entire detection system from input to output.	“http://malicious-attack.com/login”	Non-Suspicious	Non-Suspicious	Pass
2)	API Response Validation	Ensure API returns correct and timely responses for valid input.	“http://secure-portal.com”	Non-Suspicious	Error	Failed
Improved version of test case id 2)	API Response Validation	Ensure API returns correct and timely responses for valid input.	“http://secure-portal.com”	Non-Suspicious	Suspicious	Failed
3)	Frontend Display for Phishing URLs	Confirm phishing detection result is correctly displayed to	“http://unsecured-payment.com”	Suspicious	Error	Failed

		users				
Improved version of Test case id 3)	Frontend Display for Phishing URLs	Confirm phishing detection result is correctly displayed to users	“http://unsecurepayment.com”	Suspicious	Non-Suspicious	Failed
4)	Frontend Display for Legitimate URLs	Confirm legitimate detection result is correctly displayed to users.	“https://paypal.com”	Non-Suspicious	Error	Failed
Improved version of test case id 4)	Frontend Display for Legitimate URLs	Confirm legitimate detection result is correctly displayed to users..	“https://paypal.com”	Non-Suspicious	Non-Suspicious	Pass

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATION

5.1. Conclusion

This project successfully developed a Phishing Link Detection System using an LSTM-based model to identify and block phishing URLs in real time, enhancing online security by preventing identity theft and financial losses. Built on a FastAPI backend with a user-friendly interface, the system ensures accuracy and usability through a rigorous V-model methodology. While limitations like false positives and challenges with sophisticated phishing techniques exist, continuous updates and feedback integration address these gaps, providing a robust foundation for future improvements against evolving cyber threats.

5.2. Lesson learnt/ Outcome

The expected outcome of the phishing detection system is a user-friendly web page where users can input a URL for analysis. The system evaluates the entered URL and determines whether it is safe or unsafe to visit. Alongside this classification, the system provides a confidence score that reflects how certain the model is in its prediction, offering transparency and insight into the detection process.

From this project, we learned the importance of combining machine learning with a clear, accessible interface to ensure usability for non-technical users. It highlighted the need for robust feature engineering and model training to enhance accuracy and confidence in predictions. Additionally, the integration of confidence scores emphasized the value of providing users with actionable insights, fostering trust in the system while also underscoring the necessity of ongoing updates to adapt to evolving phishing tactics.

5.3. Future Recommendation

The phishing link detection system can have the following recommendations that can be implemented in future:

1. Feedback Loop & Active Learning

- Implement user feedback for false positive/negative reporting
- Create continuous model retraining pipeline with new validated data

2. Browser Extension Development

- Real-time URL checking before page loads via browser's extension
- Visual warnings through toolbar icons

3. Email Phishing Detection

- Adapt model for email analysis (headers, content, links)
- Add support for attachment scanning

4. Additional Enhancements

- Multilingual phishing detection support
- Mobile app for URL scanning

REFERENCES

- [1] S. a. S. J. Hochreiter, "Long short-term memory," *Neural computation*, vol. 9, no. 1997, pp. 1735--1780, 1997.
- [2] A. Sherstinsky, "Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) Network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 43 pages, 2020.
- [3] R. & A. Rou, "ScienceDirect," *PhishShield: A Desktop Application to Detect Phishing Webpages through Heuristic Approach*, vol. 54, no. 2015, pp. 147-156, 2015.
- [4] "easydmarc.," [Online]. Available: <https://easydmarc.com/tools/phishing-url>. [Accessed 10 August 2024].

APPENDIX

Homepage of Phishing Link Detection System

PHISHING LINK DETECTION SYSTEM

HomeAbout

Detect Phishing

Enter a URL link in format http or https and should contain www

PredictReset



Phishing Prediction on Phishing Link Detection System

PHISHING LINK DETECTION SYSTEM

HomeAbout

Detect Phishing

https://www.facebook.com

PredictedReset

Prediction: **Non-Suspicious**
Phishing Probability: **1.35%**

Proceed



Phishing prediction and Results of Phishing Link Detection System

PHISHING LINK DETECTION SYSTEM

[Home](#) [About](#)

Detect Phishing

<https://www.facebook-total-legit.com>

Predicted

Reset

Prediction: **Suspicious**

Phishing Probability: **99.12%**

Proceed With Caution



